

React Router

URLs, Pages, and Navigation

CSCI 39548 — Practical Web Development

Section Goals

- ▶ Turn a single-page app into a multi-page app with routes
- ▶ Share layout chrome with `<Outlet>`
- ▶ Build dynamic routes with URL parameters
- ▶ Drive UI state from the URL with search params
- ▶ Navigate programmatically with `useNavigate`

Section Items

- ▶ Basic Routes
- ▶ Layouts and NavLink
- ▶ Dynamic Routes and 404
- ▶ Search Params

One Page Is Not Enough

- ▶ Our app is one screen. Real apps have many: a list, a detail view, an about page, a login.
- ▶ We could fake it with `useState` (“which view am I on?”) — but then the URL never changes
- ▶ No bookmarks, no Back button, no sharing a link
- ▶ **The URL should be state.** That’s what a router gives you.

Client-Side Routing

- ▶ **Old way** (server routing): every URL = a new HTML page from the server, full reload
- ▶ **SPA way**: load the app ONCE, then JavaScript swaps which component shows based on the URL. No reload. Instant.
- ▶ React Router maps URLs → components

What We're Building

- ▶ The same Todo app becomes multi-page: Home / Todos / Todo detail / About / 404
- ▶ Router decides WHICH page
- ▶ TanStack Query still owns the DATA

Section Items

► Basic Routes

- Layouts and NavLink
- Dynamic Routes and 404
- Search Params

BrowserRouter

```
// main.tsx -- wrap the whole app
<BrowserRouter>
  <App />
</BrowserRouter>
```

Gives the app access to the URL (HTML5 history API, clean URLs).

Routes and Route

```
// App.tsx -- declare the map
<Routes>
  <Route path="/" element={<HomePage />} />
  <Route path="/todos" element={<TodosPage />} />
  <Route path="/about" element={<AboutPage />} />
</Routes>
```

“This path renders this component.” First match wins.

Link

```
<Link to="/todos">Todos</Link>
```

Navigate without reloading the page.

Link vs a href

	<code><Link to="..."></code>	<code></code>
Page reload	No	Yes
State kept	Yes	No (app reboots)
Server hit	No	Yes
Use for	Internal navigation	External links

Rule: **internal navigation** = `<Link>`. External links = `<a>`.

Routing Didn't Touch the Data

- ▶ Go to Todos (loads), to Home, back to Todos within 5s → no refetch
- ▶ The Query cache survived because the app never reloaded
- ▶ Router handles **which page**. TanStack Query handles **the data**.

Section Items

- Basic Routes

► **Layouts and NavLink**

- Dynamic Routes and 404
- Search Params

Don't Repeat the Nav

```
<Route element={<Layout />}>
  <Route index element={<HomePage />} />
  <Route path="todos" element={<TodosPage />} />
  <Route path="about" element={<AboutPage />} />
</Route>
```

- ▶ A **layout route** renders shared chrome once
- ▶ Children render inside an `<Outlet>`

The Layout Component

```
// Layout.tsx
function Layout() {
  return (
    <div>
      <nav>...</nav>
      <Outlet />    {/* matched child renders here */}
    </div>
  );
}
```

The index Route

- ▶ `<Route index>` = the default child for the parent's path
- ▶ Here, `index` under the `layout` = what shows at `/`

NavLink Highlights the Active Page

```
<NavLink to="/todos" className={({ isActive }) =>
  isActive ? "bg-blue-600 text-white" : "..."}>
  Todos
</NavLink>
```

- ▶ NavLink knows if its target is the current route
- ▶ Gives you `isActive` — no manual “which tab” tracking

The end Gotcha

```
<NavLink to="/" end>Home</NavLink>
```

- ▶ Without end, “Home” is active on EVERY route (every path starts with /)
- ▶ end = active only on an exact match

- ▶ Remove end → “Home” stuck active everywhere
- ▶ Add it back → correct behavior

Section Items

- Basic Routes
- Layouts and NavLink
- ▶ **Dynamic Routes and 404**
- Search Params

URLs That Carry Data

```
<Route path="/:id" element={<TodoDetailPage />} />
```

```
const { id } = useParams();    // "3" -- a STRING  
const todoId = Number(id);
```

- ▶ `:id` is a placeholder — matches `/todos/1`, `/todos/2`, ...
- ▶ Read it with `useParams`
- ▶ Always a string — convert with `Number()` when needed

- ▶ TodoCard's text becomes `<Link to={'/todos/${id}'}>`
- ▶ The detail page fetches that one todo + its assignee (dependent query)
- ▶ The old inline assignee panel is gone — it's a real page now

Navigating from Code

```
const navigate = useNavigate();  
  
<button onClick={() => navigate(-1)}>Back</button>
```

- ▶ `<Link>` = user clicks to navigate
- ▶ `useNavigate` = navigate from code (after submit, a Back button, a redirect)

The Catch-All 404

```
<Route path="*" element={<NotFoundPage />} />
```

- ▶ * matches anything nothing else did
- ▶ Must be the last route

Live Code: Dynamic Routes

- ▶ Click a todo → `/todos/3`
- ▶ Type `/todos/2` in the address bar → loads directly
- ▶ `/todos/999` → not-found state
- ▶ `/nonsense` → 404 page

Deep Links and the SPA Fallback

- ▶ Typing `/todos/2` works in dev because Vite serves `index.html` for any path
- ▶ On a static host you must rewrite all paths to `index.html`
- ▶ Otherwise the server 404s before React Router ever runs
- ▶ More on this in the deployment demo

Section Items

- Basic Routes
- Layouts and NavLink
- Dynamic Routes and 404

► **Search Params**

Filters Belong in the URL

```
// before
const [filter, setFilter] = useState("all");

// after
const [searchParams, setSearchParams] = useSearchParams();
const filter = searchParams.get("filter") ?? "all";
setSearchParams({ filter: "active" });
// -> /todos?filter=active
```

`useSearchParams` = `useState` for the query string.

Why Put Filters in the URL?

- ▶ **Shareable:** `/todos?filter=done` shows everyone the same view
- ▶ **Reload-safe** and bookmarkable
- ▶ **Back button** steps through filter changes
- ▶ One source of truth (the URL), not a hidden `useState`

- ▶ Value is always a **string** — whitelist it
- ▶ Drop the param for the default to keep URLs clean
- ▶ `setSearchParams(v, { replace: true })` to avoid a history entry

Live Code: Search Params

- ▶ Click filters → URL changes
- ▶ Reload on `?filter=done` → survives
- ▶ Back button → steps through filter history

What You Can Do Now

- ▶ Multi-page app with `<Routes>` + `<Link>/<NavLink>` ✓
- ▶ Shared layouts with `<Outlet>` ✓
- ▶ Dynamic routes `(:id)` + `useParams` ✓
- ▶ Programmatic nav with `useNavigate` ✓
- ▶ URL-driven state with `useSearchParams` ✓

What We Skipped (Mention Only)

- ▶ **Data router** (`createBrowserRouter`) + loaders/actions — we let TanStack Query own data instead
- ▶ Protected routes / redirects (`<Navigate>`)
- ▶ Lazy route code-splitting
- ▶ Nested layouts more than one level deep

Before Next Class

- ▶ Add a `/todos/:id/edit` route to the app, reachable from the detail page